Either & Railway-Oriented Programming

son: 6 · FP Foundations · Category: Functional · Level: Intermediate · Build with Wa

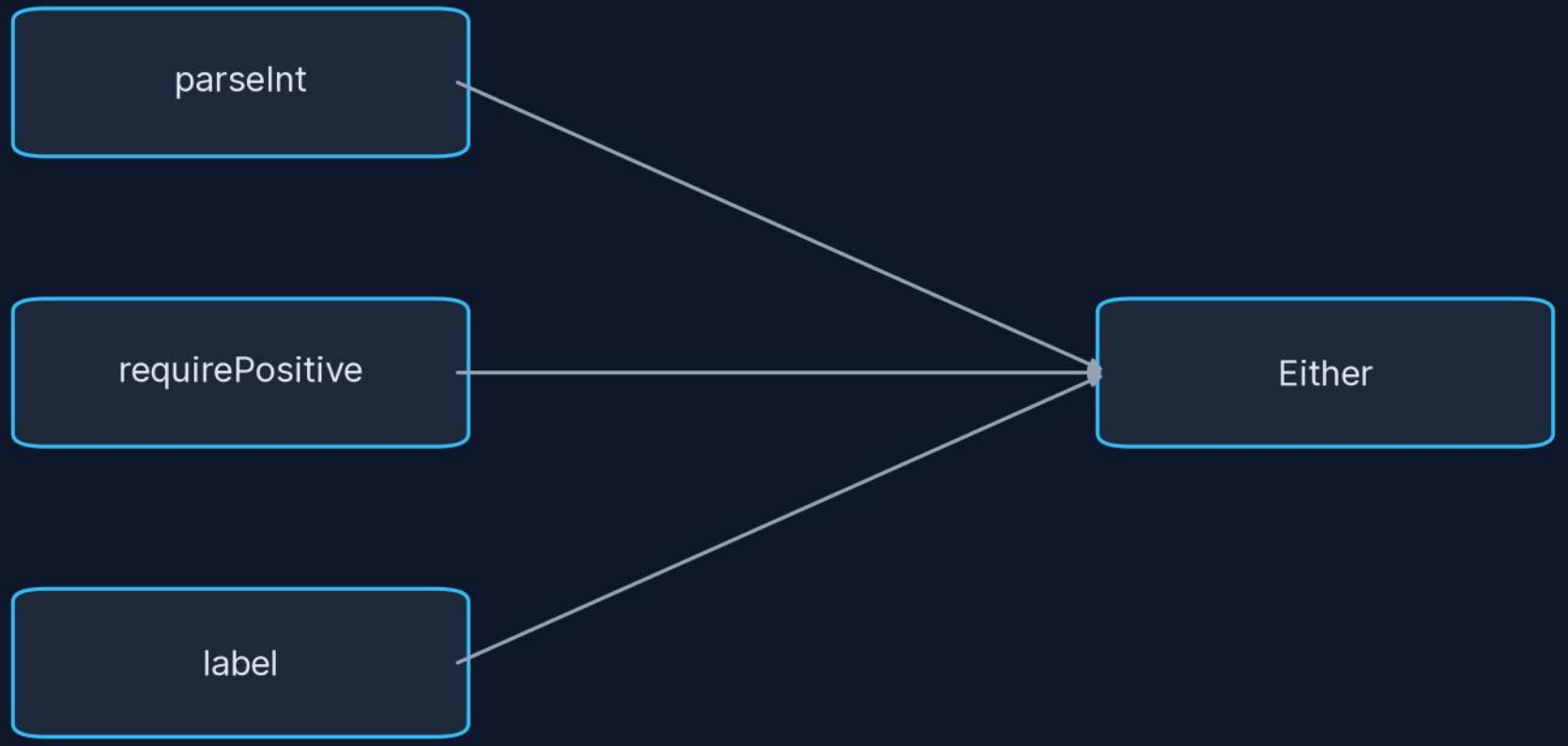
Today's Hook

- Today: Either & Railway-Oriented Programming as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Either & Railway-Oriented Programming structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Either & Railway-Oriented Programming
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/either/Railway.java
package com.buildwithwaqar.patterns.functional.either;

import dev.zazr.control.Either;

/** Railway-oriented programming with Zazr Either. */
public final class Railway {
    private Railway() {}

    public static Either<String, Integer> parseInt(String raw) {
        try {
            return Either.right(Integer.parseInt(raw));
        } catch (NumberFormatException e) {
            return Either.left("not an int: " + raw);
        }
    }

    public static Either<String, Integer> requirePositive(int n) {
        return n > 0 ? Either.right(n) : Either.left("not positive: " + n);
    }

    public static Either<String, String> label(String raw) {
        return parseInt(raw)
                .flatMap(Railway::requirePositive)
                .map(n -> "id-" + n);
    }
```

Implementation (2)

```
    public static String explain(Either<String, String> result) {
        return switch (result) {
            case Either.Right(var v) -> "ok:" + v;
            case Either.Left(var err) -> "err:" + err;
        };
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.functional.either;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Either / Railway walkthrough ===");
    System.out.println(Railway.explain(Railway.label("7")));
    System.out.println(Railway.explain(Railway.label("x")));
    System.out.println(Railway.explain(Railway.label("-3")));
  }
}
```

Walkthrough Output

```
=== Either / Railway walkthrough ===
ok:id-7
err:not an int: x
err:not positive: -3
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Either & Railway-Oriented Programming
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Either & Railway-Oriented Programming → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.